

HEALTH INSURANCE POLICY & CLAIMS MANAGEMENT SYSTEM

PROJECT REPORT

Database Management Systems

Student Name	Tanush Reddy Vanga
Roll Number	25WU0102287
Course	B.Tech CSE (AI & ML)
Faculty Name	Ms Kiran Mayee
Academic Year	2026–2027

Submitted as part of the DBMS Course Project.

2. Abstract

The Health Insurance Policy and Claims Management System is a relational database application for managing customers, insurance policies, hospitals, treatments, claims and claim payments. MySQL is used as the database backend and Python Tkinter provides the graphical interface. The system demonstrates insertion, viewing, deletion, search and relational retrieval using JOIN queries.

The database contains eight core tables: Customer, Insurance_Company, Policy_Type, Policy, Hospital, Treatment, Claim and Claim_Payment. Primary keys uniquely identify records, while foreign keys establish relationships and preserve referential integrity. Python communicates with MySQL through MySQL Connector/Python and uses parameterized SQL for user-driven operations.

3. Introduction and Problem Statement

Health insurance systems manage several related entities. A customer can hold policies, policies are issued by insurance companies and belong to policy types, and claims are associated with hospitals and treatments. Without a structured relational database, this information can become difficult to retrieve and maintain.

Problem Statement

The project addresses the need for a centralized database that stores health-insurance information and allows users to perform common database operations through a graphical interface while preserving valid relationships between records.

Proposed Solution

A MySQL relational database was designed around eight entities. A Python Tkinter interface was connected to MySQL. The application provides customer insertion, deletion and viewing, together with policy and claim views that retrieve related information using JOIN queries.

4. Objectives and Scope

Objectives

- Design a relational database for health insurance policy and claim management.
- Use primary keys, foreign keys and constraints to maintain data integrity.
- Implement INSERT, SELECT and DELETE operations.
- Use JOIN queries to display related information across tables.
- Develop a Python Tkinter interface connected to MySQL.
- Demonstrate insertion, deletion and viewing through the UI.
- Document the project using GitHub.

Scope

- Customer, insurance company and policy-type management.
- Insurance policy creation and viewing.
- Hospital and treatment information.
- Claim and claim-payment data structure.
- Graphical database interaction and live database verification.

5. Software and Hardware Requirements

Software Requirements

Component	Requirement
Operating System	Windows / compatible desktop OS
Database	MySQL
Programming Language	Python
GUI Framework	Tkinter
Database Driver	MySQL Connector/Python
IDE	Visual Studio Code
Version Control	Git and GitHub

Hardware Requirements

Component	Requirement
Processor	Modern dual-core processor or better
RAM	4 GB minimum; 8 GB recommended
Storage	At least 1 GB free
Display	Standard desktop/laptop display
Network	Internet access for GitHub and package installation

6. ER Diagram

The ER diagram models the eight core entities and their relationships: Customer–Policy, Insurance_Company–Policy, Policy_Type–Policy, Policy–Claim, Hospital–Claim, Treatment–Claim and Claim–Claim_Payment. It identifies attributes, primary/foreign keys and relationship cardinalities.

Relationships: The ER Diagram

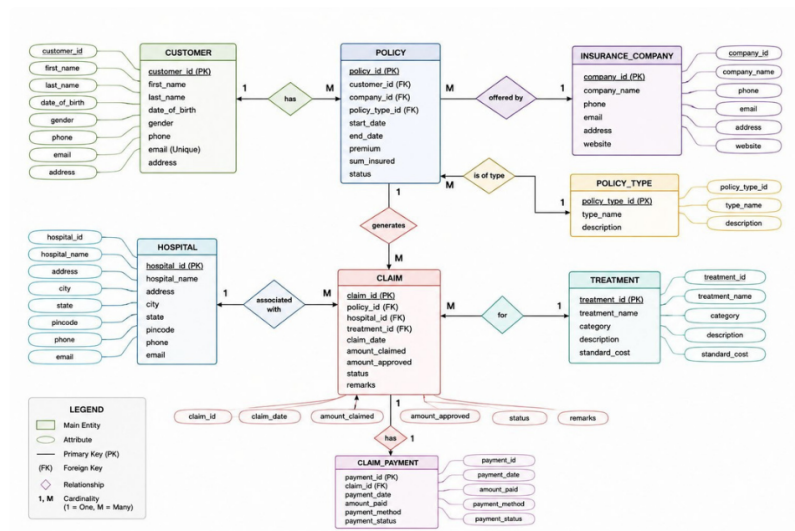


Figure 1. Entity–Relationship diagram used for the project.

7. Relational Schema and Normalization

Relational Schema

Table	Primary Key	Foreign Keys
Customer	customer_id	—
Insurance_Company	company_id	—
Policy_Type	policy_type_id	—
Policy	policy_id	customer_id, company_id, policy_type_id
Hospital	hospital_id	—
Treatment	treatment_id	—
Claim	claim_id	policy_id, hospital_id, treatment_id
Claim_Payment	payment_id	claim_id

Normalization

The schema uses separate entity tables with atomic attributes, supporting First Normal Form (1NF). Each non-key attribute is stored with the key of its own table, supporting Second Normal Form (2NF). Company, policy-type, hospital and treatment details are kept in separate tables rather than being repeated in Policy or Claim, reducing transitive dependencies and supporting Third Normal Form (3NF).

8. Data Dictionary

The following compact data dictionary summarizes the implemented table structures without forcing every table onto a separate page.

Customer

Attribute	Data Type	Constraint
customer_id	INT	PK, AUTO_INCREMENT
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
date_of_birth	DATE	NOT NULL
gender	VARCHAR(20)	—
phone	VARCHAR(15)	—
email	VARCHAR(100)	UNIQUE
address	VARCHAR(255)	—

Insurance_Company

Attribute	Data Type	Constraint
company_id	INT	PK, AUTO_INCREMENT
company_name	VARCHAR(100)	NOT NULL
phone	VARCHAR(15)	—
email	VARCHAR(100)	—
address	VARCHAR(255)	—
website	VARCHAR(150)	—

Policy_Type

Attribute	Data Type	Constraint
policy_type_id	INT	PK, AUTO_INCREMENT
type_name	VARCHAR(50)	NOT NULL
description	VARCHAR(255)	—

Policy

Attribute	Data Type	Constraint
policy_id	INT	PK, AUTO_INCREMENT
customer_id	INT	FK, NOT NULL
company_id	INT	FK, NOT NULL
policy_type_id	INT	FK, NOT NULL
start_date / end_date	DATE	NOT NULL; end_date > start_date
premium	DECIMAL(12,2)	NOT NULL; > 0
sum_insured	DECIMAL(15,2)	NOT NULL; > 0
status	VARCHAR(20)	NOT NULL

Hospital

Attribute	Data Type	Constraint
hospital_id	INT	PK, AUTO_INCREMENT
hospital_name	VARCHAR(100)	NOT NULL
address / city / state	VARCHAR	—
pincode	VARCHAR(10)	—
phone	VARCHAR(15)	—
email	VARCHAR(100)	—

Treatment

Attribute	Data Type	Constraint
treatment_id	INT	PK, AUTO_INCREMENT
treatment_name	VARCHAR(100)	NOT NULL
category	VARCHAR(100)	—
description	VARCHAR(255)	—
standard_cost	DECIMAL(12,2)	CHECK >= 0

Claim

Attribute	Data Type	Constraint
claim_id	INT	PK, AUTO_INCREMENT
policy_id / hospital_id / treatment_id	INT	FK, NOT NULL
claim_date	DATE	NOT NULL
amount_claimed	DECIMAL(15,2)	NOT NULL; > 0
amount_approved	DECIMAL(15,2)	DEFAULT 0; <= claimed
status	VARCHAR(20)	NOT NULL
remarks	VARCHAR(255)	—

Claim_Payment

Attribute	Data Type	Constraint
payment_id	INT	PK, AUTO_INCREMENT
claim_id	INT	FK, UNIQUE, NOT NULL
payment_date	DATE	—
amount_paid	DECIMAL(15,2)	NOT NULL; > 0
payment_method	VARCHAR(30)	—
payment_status	VARCHAR(20)	NOT NULL

9. SQL Commands Used (DDL and DML)

DDL — Data Definition Language

```
CREATE DATABASE IF NOT EXISTS health_insurance;  
USE health_insurance;
```

```
CREATE TABLE Customer (  
    customer_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    date_of_birth DATE NOT NULL,  
    email VARCHAR(100) UNIQUE  
);
```

DML — Data Manipulation Language

```
INSERT INTO Customer  
(first_name, last_name, date_of_birth, gender, phone, email, address)  
VALUES  
( 'Rahul', 'Sharma', '2002-05-14', 'Male',  
  '9876543210', 'rahul.sharma@email.com', 'Hyderabad' );  
  
SELECT * FROM Customer;  
  
DELETE FROM Customer  
WHERE customer_id = ?;
```

Foreign-Key Constraint Example

```
FOREIGN KEY (customer_id)  
REFERENCES Customer(customer_id)
```

10. Queries with Outputs

Query 1 — Customer and Policy Details

```
SELECT c.customer_id,
       CONCAT(c.first_name,' ',c.last_name) AS customer_name,
       p.policy_id, p.premium, p.sum_insured, p.status
FROM Customer c
JOIN Policy p
ON c.customer_id = p.customer_id;
```

Purpose: combines Customer and Policy so the application can display the customer name with policy information.

Query 2 — Customer, Policy and Claim Details

```
SELECT c.first_name, c.last_name, p.policy_id, cl.claim_id,
       cl.amount_claimed, cl.amount_approved, cl.status
FROM Customer c
JOIN Policy p ON c.customer_id = p.customer_id
JOIN Claim cl ON p.policy_id = cl.policy_id;
```

Purpose: combines three related tables to retrieve claim information together with the customer and policy.

Sample Output

Customer	Policy	Claim	Claimed	Approved	Status
Rahul Sharma	1	1	120000	95000	Approved
Ananya Rao	2	2	75000	0	Pending
Priya Nair	4	3	250000	200000	Approved
Karan Verma	5	4	50000	0	Rejected
Rahul Sharma	1	5	90000	85000	Approved

11. UI Design and Screenshots

The Python Tkinter application provides the graphical layer over the MySQL database. The screenshots document the implemented screens.

Dashboard

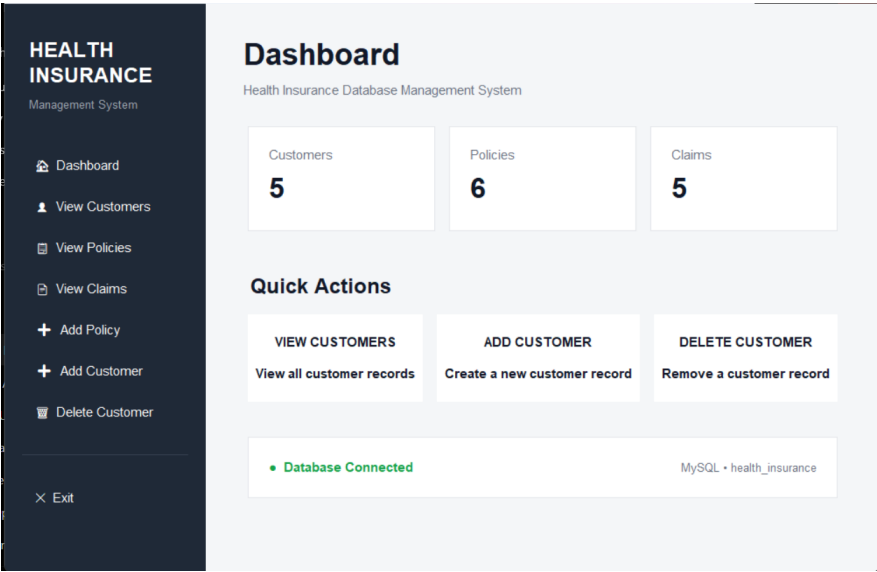


Figure 2. Dashboard with live database counts and connection status.

Customer Records

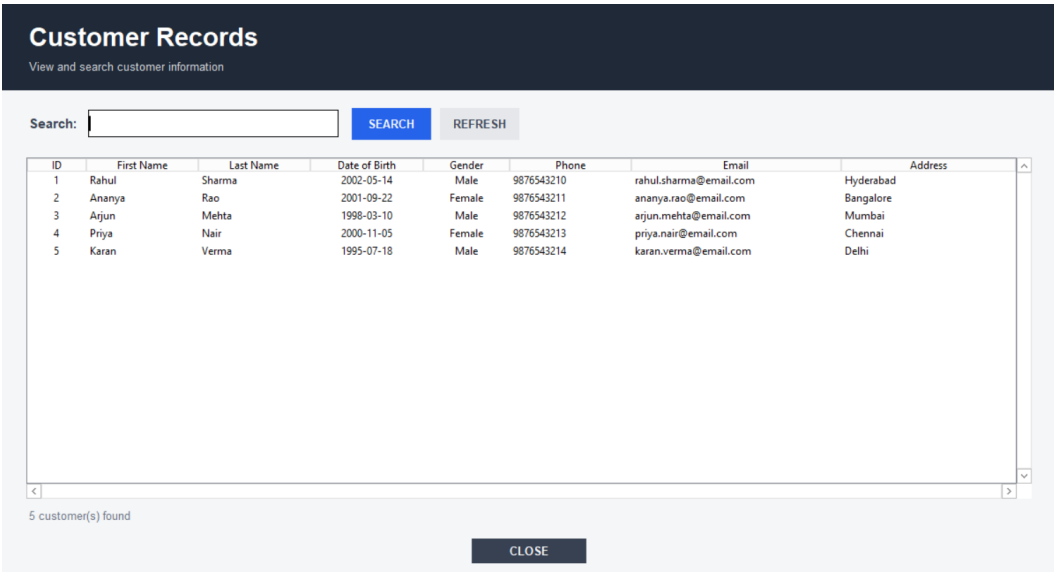


Figure 3. Customer records and search.

Add Customer

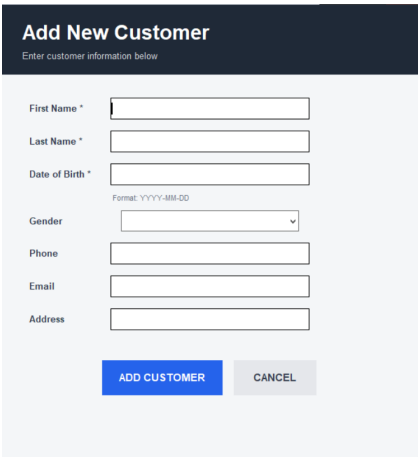


Figure 4. Customer insertion form.

Delete Customer

Delete Customer

Remove an existing customer record

Customer ID

Enter the ID of the customer you want to delete.

DELETE CUSTOMER

CANCEL

Figure 5. Customer deletion interface.

Policy Records

Policy Records

View active and historical insurance policies

Search:

SEARCH

REFRESH

Policy ID	Customer	Company	Policy Type	Start Date	End Date	Premium	Sum Insured	Status
1	Rahul Sharma	HealthSecure Insurance	Individual	2026-01-01	2026-12-31	25000.00	1000000.00	Active
2	Ananya Rao	HealthSecure Insurance	Family	2026-02-15	2027-02-14	32000.00	1500000.00	Active
3	Arjun Mehta	MediCare Insurance	Individual	2025-06-01	2026-05-31	18000.00	500000.00	Expired
4	Priya Nair	MediCare Insurance	Senior Citizen	2026-03-10	2027-03-09	45000.00	2000000.00	Active
5	Karan Verma	LifeShield Health	Family	2026-04-01	2027-03-31	28000.00	1000000.00	Active
6	Rahul Sharma	HealthSecure Insurance	Individual	2026-10-01	2027-09-30	25000.00	1000000.00	Active

6 policy/policies found

CLOSE

Figure 6. Policy records.

Claim Records

Claim Records

View and search health insurance claims

Search:

SEARCH

REFRESH

Claim ID	Customer	Policy ID	Hospital	Treatment	Claim Date	Claimed	Approved	Status	Remarks
1	Rahul Sharma	1	Apollo Hospital	Heart Surgery	2026-05-10	120000.00	95000.00	Approved	Hospitalization claim approved
2	Ananya Rao	2	Care Hospital	Appendectomy	2026-06-18	75000.00	0.00	Pending	Documents under verification
3	Priya Nair	4	Apollo Hospital	Knee Replacement	2026-07-05	250000.00	200000.00	Approved	Partial claim approved
4	Karan Verma	5	Fortis Hospital	Heart Surgery	2026-08-12	50000.00	0.00	Rejected	Treatment not covered under policy
5	Rahul Sharma	1	Care Hospital	Appendectomy	2026-09-01	90000.00	85000.00	Approved	Claim approved after review

5 claim(s) found

Figure 7. Claim records with related information.

Add Policy

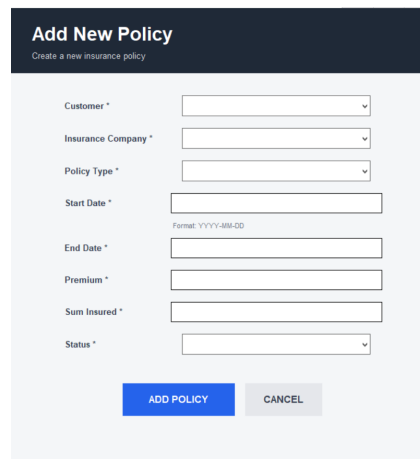


Figure 8. Policy insertion interface.

12. Implementation Details

Layer	Technology	Role
Frontend	Python Tkinter	Graphical interface
Application	Python	Validation, event handling and SQL
Connector	MySQL Connector/Python	Python–MySQL connectivity
Backend	MySQL	Persistent relational storage
Version Control	Git / GitHub	Source and documentation

Database Connectivity

```
db = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="YOUR_MYSQL_PASSWORD",  
    database="health_insurance"  
)
```

The local application connects to the health_insurance database. The actual password should not be committed to a public GitHub repository.

Parameterized SQL

```
query = "DELETE FROM Customer WHERE customer_id = %s"  
cursor.execute(query, (customer_id,))
```

Parameterized queries keep user values separate from the SQL statement and help prevent SQL injection.

UI-to-Database Flow

User action → Tkinter event → validation → MySQL connection → SQL query → database result/commit → UI refresh or feedback.

13. Testing — Test Cases and Results

Test	Action	Expected Result	Result
TC01	Open application	Dashboard + connected status	Pass
TC02	View Customers	Records displayed	Pass
TC03	Add valid customer	New record inserted	Pass
TC04	Search customer	Matching records shown	Pass
TC05	Delete valid customer	Record removed	Pass
TC06	View Policies	Policy records displayed	Pass
TC07	View Claims	Claim records displayed	Pass
TC08	Invalid FK	MySQL rejects invalid reference	Pass
TC09	Invalid input	UI validation blocks bad input	Pass
TC10	Refresh	Latest database state shown	Pass

During implementation, MySQL Error 1452 was encountered when a child record referenced a parent ID that did not exist. This confirmed that foreign-key constraints were active. The solution was to insert required parent records before inserting child records.

Recommended live demonstration: View existing records → insert a temporary customer → verify the new record → delete it → verify deletion → show Policy and Claim views.

14. Conclusion and Future Enhancements

The project successfully connects relational database design with a working graphical application. The eight-table MySQL database stores the major entities of a health insurance workflow, while primary keys, foreign keys and constraints preserve valid relationships. The Python Tkinter application provides practical database interaction through customer insertion, viewing and deletion, together with relational Policy and Claim views.

Future Enhancements

- Authentication and role-based access control.
- Full CRUD workflows for claims and claim payments.
- Policy and claim update functions.
- Reporting dashboards for policy and claim analytics.
- Centralized environment-based database configuration.
- Deployment to a controlled server environment.

15. References

- MySQL documentation and SQL reference materials.
- Python documentation for Tkinter and database connectivity.
- Project-provided DBMS presentation and ER-diagram materials.
- Course lectures and laboratory exercises on relational databases, SQL, keys, constraints and JOINS.
- GitHub documentation for Git and repository management.

16. Appendix — GitHub Repository Link

Repository: <https://github.com/TanushReddy-9/DBMS-Course-Project>

The official project instructions require a public repository named DBMS-Course-Project containing Presentation-I, Presentation-II, Presentation-III and Project-Report folders. The project report is required to be uploaded by 10 October 2026.

Suggested Repository Structure

```
DBMS-Course-Project/  
  README.md  
  Presentation-I/  
  Presentation-II/  
  Presentation-III/  
    main.py  
    requirements.txt  
    Screenshots/  
  Project-Report/  
    Health_Insurance_Project_Report.pdf
```

Final Submission Checklist

- Cover page completed with student name, roll number, course, faculty name and academic year.
- ER diagram included.
- Relational schema and normalization included.
- Data dictionary included.
- DDL/DML commands and sample outputs included.
- Queries and outputs included.
- UI screenshots included.
- Implementation details included.
- Testing results included.
- Conclusion and future enhancements included.
- GitHub repository link included.

— End of Report —